

MiniMax H3：通用全模态音视频生成模型

导论

MiniMax H3 是 MiniMax 在 2026 年发布的通用全模态生成系统。它能够读取文本、图片、视频和音频条件，并在一次生成过程中同时得到视频与双声道音频。完整系统包含 H3-Context-IR、H3-Base 和 H3-Regenerate-2K。当前能够从公开权重和代码中完整分析的是 H3-Base；H3-Context-IR 负责把复杂输入整理成结构化上下文，H3-Regenerate-2K 负责在基础结果上继续生成 2K 输出，这两部分目前仍以托管系统为主。

H3-Base 的主干是 H3-Omni-Transformer。它把文本、视频 latent 和音频 latent 都变成 5376 维 Token，再写入同一条 Packed Sequence。50 层 Transformer 始终处理这一条统一序列，因此文本、画面和声音可以在每一层 Self-Attention 中直接交换信息。不同模态仍然保留自己的输入投影、位置坐标、噪声时间、AdaLN 调制参数和输出头，所以统一序列不会丢失模态身份。

H3 的视频和音频都在 latent 空间中生成。VisualVAE 先把视频压缩为 24 通道时空 latent，再经过 $1 \times 2 \times 2$ Patchify 变成视频 Token；AudioVAE 把 32 kHz 波形压缩成 40 Hz、32 通道音频 latent。生成阶段使用 Rectified Flow，视频和音频各有一条独立时间表，但两种模态在每个采样步共享同一次 Transformer 前向，因此同步关系可以在整个去噪过程里持续建立。

这一课重点解释 H3-Base 的完整数据流，并把每个模块的数学来源讲清楚：为什么需要 VAE、为什么要 Patchify、为什么统一序列还需要 MM-RoPE、为什么 AdaLN 同时读取时间和模态、Self-Attention 怎样实现跨模态交互，以及 Rectified Flow 怎样一步一步把随机 latent 变成最终视频和音频。

官方仓库：<https://github.com/MiniMax-AI/MiniMax-H3>

官方模型页：<https://huggingface.co/MiniMaxAI/MiniMax-H3>

一、先建立 H3-Base 的整体数据流

H3-Base 需要处理三类信息。第一类是文本和高层视觉条件，它们由 Qwen3-VL-32B 编码；第二类是视频 latent，它们由 H3-VisualVAE 产生；第三类是音频 latent，它们由 H3-AudioVAE 产生。三类表示进入主干以前维度不同：文本条件是 5120 维，视频 Patch 是 96 维，音频 latent 是 32 维。H3 为每一种输入准备独立线性投影，最终统一到 5376 维。

统一维度以后，模型还不会立刻送进 Transformer。Pipeline 会为文本、视频和音频准备三组位置索引，把它们写入同一个隐藏矩阵 $H^{(0)} \in \mathbb{R}^{N \times 5376}$ 。这里 N 是总

Token 数。每个 Token 还携带三类附加信息：模态标签 m_i 、噪声时间索引 r_i 和三维位置 $p_i = (t_i, h_i, w_i)$ 。这三类信息分别告诉模型“当前 Token 属于哪种模态”“它现在处于哪个去噪阶段”“它在时间和空间中的位置”。

50 层 H3-Omni-Transformer 始终保持隐藏宽度 5376 不变。每层先通过 AdaLN 调整归一化后的表示,再做统一 Self-Attention;随后再经过第二次 AdaLN 和 SwiGLU FFN。最后一个 Final AdaLN 处理整条序列, 视频输出头把视频位置映射成 96 维 velocity, 音频输出头把音频位置映射成 32 维 velocity。两个 scheduler 再分别更新视频和音频 latent, 进入下一采样步。

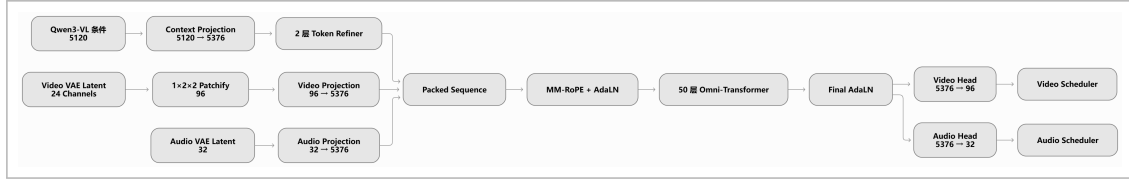


图 1 H3-Base 从多模态条件到联合音视频 velocity 的整体数据流。

二、为什么主干参数量会达到约 33B

公开配置中, H3-Omni-Transformer 有 50 层, 隐藏维度为 5376, Attention 使用 56 个 Head, 每个 Head 为 128 维, 所以 Attention 内部宽度是 7168。FFN 使用 SwiGLU, 中间维度为 14336。仅从这几个维度就能看出单层参数量很大。

一层 Attention 有 Q、K、V 三个输入投影和一个输出投影。因为输入宽度是 5376, 内部宽度是 7168, 所以一层 Attention 约有 $3 \times 5376 \times 7168 + 7168 \times 5376 \approx 154.1\text{M}$ 个权重。SwiGLU 有 Gate、Up 和 Down 三个矩阵, 参数量约为 $3 \times 5376 \times 14336 \approx 231.2\text{M}$ 。

AdaLN 的参数规模更大。时间条件经过 Timestep Embedding 后是 2688 维, 每一层都需要为 video、text、audio 三种模态生成 Attention 和 FFN 的 shift、scale、gate, 共 6 组参数。每组长度都是 5376, 所以 AdaLN 的输出宽度是 $6 \times 5376 \times 3 = 96768$ 。单层 AdaLN Projection 约有 $2688 \times 96768 \approx 260.1\text{M}$ 个权重, 50 层累计约 13B。把 Attention、FFN 和 AdaLN 加在一起, 一层大约 645M, 50 层约 32B, 再加上 Text Refiner、输入输出投影和 Final AdaLN, 整体落在官方公布的约 33B 范围内。

项目	配置
H3-Omni-Transformer	约 33B Dense
AdaLN 相关参数	约 13B
主干层数	50
隐藏维度	5376
Attention Heads	56
Head Dimension	128
Attention Inner Dimension	7168
FFN Dimension	14336
Text Token Refiner	2 层
Video Latent Channels	24
Audio Latent Channels	32
Video Patch	$1 \times 2 \times 2$
Text Encoder Output	5120
Time Embedding Output	2688
MM-RoPE Frequency Dimension	16 / axis

这里需要注意，像 $56 \times 128 = 7168$ 、 $6 \times 5376 \times 3 = 96768$ 这种纯数字维度计算都应该放在正文中理解，它们只是说明张量为什么具有这个宽度，没有必要单独占一行。

三、H3-Encoder：为什么使用 Qwen3-VL 的中间隐藏状态

H3-Encoder 使用 Qwen3-VL-32B 的完整预训练权重。文本、图片和参考视频先经过 Qwen3-VL 的多模态理解网络，H3 读取第 50 层隐藏状态作为条件表示。公开代码直接取 `hidden_states[50]`，得到的特征宽度为 5120。

这里使用中间隐藏状态的原因可以从生成模型需要的信息类型理解。语言模型最后的 LM Head 负责把隐藏状态映射成词表概率，而 H3 并不需要预测下一个文本 Token。它需要的是已经经过多层多模态交互后的连续语义表示，因此直接取 Decoder 中间隐藏状态更合适。设得到的条件特征为 $C \in \mathbb{R}^{N_t \times 5120}$ ，H3 再通过 Context Projection 得到 $C_0 = CW_c + b_c \in \mathbb{R}^{N_t \times 5376}$ ，其中 $W_c \in \mathbb{R}^{5120 \times 5376}$ 。

视觉参考会同时走两条路径。Qwen3-VL 给出高层语义表示，用来描述对象、场景、动作和文字要求；VisualVAE 把同一份图片或视频编码到 H3 自己的生成 latent 空间。前者提供理解层面的条件，后者提供和目标视频处于同一 latent 空间的视觉条件。两种信息后续都能进入统一序列。

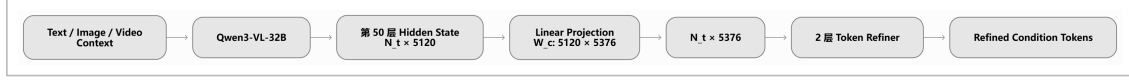


图 2 H3-Encoder 从 Qwen3-VL 第 50 层得到条件表示。

四、Text Token Refiner：为什么还要再处理两层

Qwen3-VL 输出的特征已经包含丰富语义，但这些特征来自“理解模型”的表示空间。H3 主干接下来要做的是音视频生成，因此条件表示还需要适配生成网络的数值分布和隐藏维度。Context Projection 只解决维度问题，Text Token Refiner 进一步让条件 Token 在 H3 自己的隐藏空间中完成两轮 Self-Attention 和 FFN 更新。

设第 r 层输入为 $C^{(r)} \in \mathbb{R}^{N_t \times 5376}$ 。先进行 Pre-Norm Self-Attention：

$$U^{(r)} = C^{(r)} + \text{Attn}(\text{RMSNorm}(C^{(r)})) .$$

随后再做 Pre-Norm SwiGLU：

$$C^{(r+1)} = U^{(r)} + \text{FFN}(\text{RMSNorm}(U^{(r)})) .$$

Refiner 的 Attention 仍然是 56 个 Head、每个 Head 128 维，FFN 维度仍然是 14336。两层结束后再做一次 RMSNorm。这里没有主干 AdaLN，因为 Refiner 只处理条件 Token，还没有进入视频和音频的 Flow 去噪过程；也没有主干 MM-RoPE，因为它不负责建立完整音视频时空坐标。

五、VisualVAE：视频为什么一定要先进入 latent 空间

直接让 Transformer 处理 RGB 视频会产生很长的序列。假设视频有 T 帧，每帧分辨率为 $H \times W$ ，即使只把每个像素位置看成一个 Token，序列长度也与 THW 成正比。Self-Attention 的主要计算量随 Token 数量平方增长，所以高分辨率长视频必须先压缩。

H3-VisualVAE 把视频编码成 24 通道 latent。公开实现是 KL-VAE，Encoder 最后输出 48 个通道，其中一半对应均值 $\mu_\phi(x)$ ，另一半对应方差参数。对某个时空位置，可以写成

$$q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \text{diag}(\sigma_\phi^2(x))) .$$

训练阶段常用重参数化 $z = \mu + \sigma \odot \epsilon$ ，其中 $\epsilon \sim \mathcal{N}(0, I)$ 。这样随机性放在 ϵ 中， μ 和 σ 仍然可以通过反向传播学习。H3 公开推理实现确实使用 `DiagonalGaussianDistribution` 表示这一 posterior；官方当前没有完整披露 VAE 训练时重建损失、KL 损失和其他损失的精确权重，因此这里不补充未公开数值。

H3-VisualVAE 的空间压缩倍率是 16，时间压缩倍率是 4，所以一个尺寸约为 $T \times H \times W$ 的视频会被压成 $24 \times T' \times H' \times W'$ ，其中 $T' \approx T/4$ 、 $H' \approx H/16$ 、 $W' \approx W/16$ 。实际时间长度还要按照因果卷积和官方帧数规则处理。

VisualVAE 使用因果 3D 卷积。空间卷积可以同时读取邻近像素，时间卷积只读取当前与过去帧，不读取未来帧。这样在处理长视频或分块编码时，当前 latent 不会依赖未来内容，时间边界更容易保持一致。

进入 H3 主干以前，latent 还会按通道做标准化。如果第 c 个 latent 通道在训练数据上的统计量是 μ_c, σ_c ，则送入生成主干的是 $\tilde{z}_c = (z_c - \mu_c)/\sigma_c$ 。解码前再恢复为 $z_c = \tilde{z}_c \sigma_c + \mu_c$ 。H3 对 24 个通道分别保存 mean/std，这样不同通道的数值范围可以被单独校正。

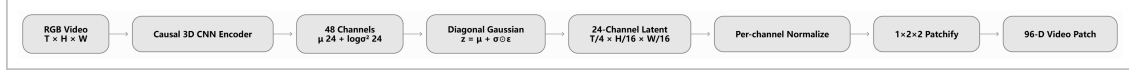


图 3 H3-VisualVAE 的高斯 latent、因果时空压缩与通道标准化。

六、VisualVAE Decoder：为什么编码器和解码器结构不同

H3 的 VisualVAE Encoder 主要使用因果 3D CNN，而 Decoder 使用 36 层非因果 ViT。这个差异与两个阶段的任务不同。Encoder 面对的是长视频输入，需要稳定地压缩局部时空信息，并支持因果分块；Decoder 已经拿到完整 latent 视频，可以一次性读取更大范围的时空关系，再恢复像素细节。

公开 Decoder 先把每个 24 维 latent voxel 投影到 2048 维，然后加入 4 个可学习 Register Token 和一个零 Token，再进入 36 层 ViT。每层使用 32 个 Attention Head，每个 Head 64 维。Transformer 处理结束后，每个有效 latent Token 被投影为一个 RGB 像素块。

由于 VisualVAE 的空间压缩倍率为 16、时间压缩倍率为 4，一个 latent voxel 最终需要恢复 $4 \times 16 \times 16$ 个 RGB 像素位置，因此输出宽度是 $3 \times 4 \times 16 \times 16 = 3072$ 。Decoder 最后把这些 3072 维向量按照时间和空间重新排列成连续视频。

这部分说明 H3 的视频 latent 同时保留空间和时间信息。解码阶段通过时空 Transformer 恢复多帧像素，因此后续生成主干可以直接在时空 latent 上学习运动变化。

七、为什么 VAE 后面还要做一次 $1 \times 2 \times 2$ Patchify

VisualVAE 已经把空间缩小 16 倍、时间缩小 4 倍，但对 768p 或更长视频来说，latent 网格仍然很大。H3 在送入 Omni-Transformer 前再做一次 $1 \times 2 \times 2$ Patchify，进一步减少视频 Token 数。

每个 Patch 包含 24 个 latent 通道、1 个时间位置、 2×2 个空间位置，所以展平后宽度是 96。这个 96 维向量经过 Video Projection 映射到 5376 维。由于时间方向

没有再合并，时间压缩仍然是 4；空间方向又合并 2 倍，因此最终进入 Transformer 时，相对原视频的有效空间压缩倍率是 32。

如果 VAE latent 大小为 $T' \times H' \times W'$ ，那么视频 Token 数约为 $N_v = T'(H'/2)(W'/2)$ 。这一步直接把视觉 Self-Attention 的序列长度缩小约 4 倍。它不改变 latent 的语义，只把邻近 2×2 空间位置合成一个更宽的输入向量，再交给线性层统一到 5376 维。

八、AudioVAE：从 32 kHz 波形到 40 Hz latent

音频如果直接按 32 kHz 采样率送入 Transformer，一秒就有 32000 个时间点，序列长度非常大。H3-AudioVAE 通过多级下采样把波形压缩为 40 Hz latent。公开配置的下采样倍率依次是 2、4、4、5、5，总倍率为 800，所以 32000 Hz 音频经过编码以后，每秒只剩 $32000/800 = 40$ 个 latent 时间位置。

每个 latent Token 有 32 个通道。左右声道使用同一套 AudioVAE 权重，但分别编码和解码。这样既共享了声学表示，又保留左右声道各自的信息。进入 Omni-Transformer 前，每个 32 维音频 latent 再通过 Audio Projection 变成 5376 维。

视频最终是 24 FPS，音频 latent 是 40 Hz，所以两种模态的时间网格不同。H3 给两种 Token 分配共享时间坐标，再由统一 Self-Attention 学习它们之间的对应关系。这样一个视频动作可以对应多个音频 Token，一段持续声音也可以覆盖多个视频帧。

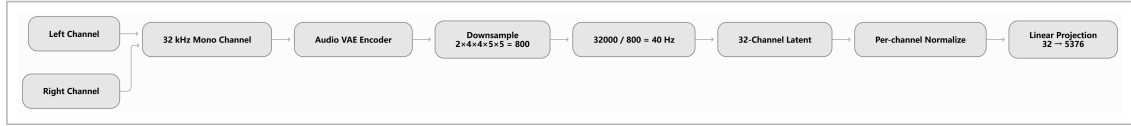


图 4 H3-AudioVAE 从 32 kHz 波形压缩为 40 Hz、32 通道 latent。

九、Packed Sequence：三种模态怎样放进同一个矩阵

经过投影以后，文本、视频和音频都已经是 5376 维。设文本矩阵为 $H_t \in \mathbb{R}^{N_t \times 5376}$ ，视频矩阵为 $H_v \in \mathbb{R}^{N_v \times 5376}$ ，音频矩阵为 $H_a \in \mathbb{R}^{N_a \times 5376}$ 。Pipeline 另外提供三组索引 I_t, I_v, I_a ，用于指定三种 Token 在统一序列中的位置。

先初始化一个总隐藏矩阵 $H^{(0)} \in \mathbb{R}^{N \times 5376}$ ，其中 $N = N_t + N_v + N_a$ 。然后按索引写入：

$$H^{(0)}[I_t] = H_t, \quad H^{(0)}[I_v] = H_v, \quad H^{(0)}[I_a] = H_a.$$

这个矩阵就是 Single-Stream 的实际输入。后面 50 层主干始终处理 $H^{(\ell)} \in \mathbb{R}^{N \times 5376}$ ，不会再拆成三个独立网络。任意两个位置之间的联系都通过同一个 Attention Matrix 建立。

为了让模型知道每个位置来自什么模态,公开实现给每个 Token 一个 tag: video 为 0, text 为 1, audio 为 2。除此之外,每个 Token 还保存 timestep index 和 (t, h, w) 坐标。模态标签交给 AdaLN, timestep index 决定当前去噪阶段,三维坐标交给 MM-RoPE。这样统一序列在共享计算的同时仍然保留模态、噪声时间和时空位置三类结构信息。

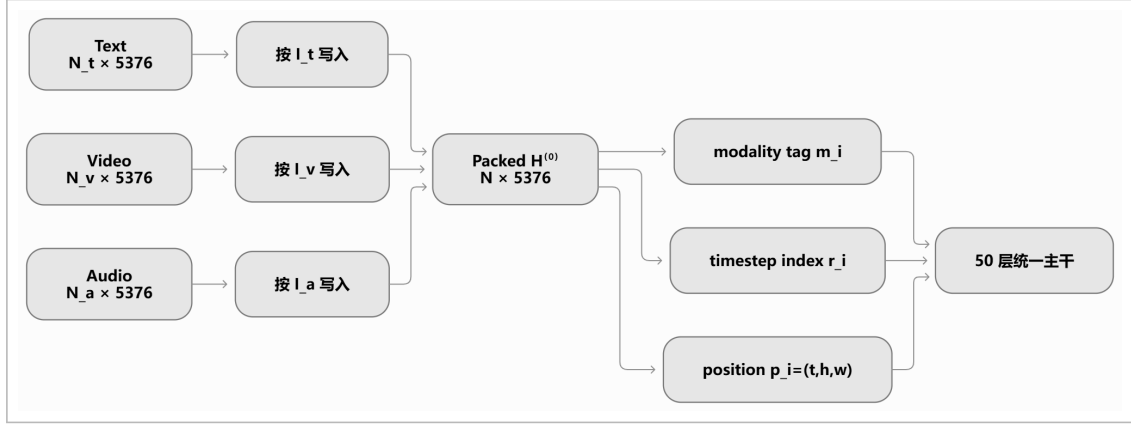


图 5 三种模态通过索引写入统一 Packed Sequence。

十、MM-RoPE：为什么一维 Packed Sequence 还能保留视频时空结构

Transformer 实际处理的是一维序列,视频原本却有时间、高度、宽度三个坐标。H3 为每个 Token 保存 $p = (t, h, w)$, 并在 Q、K 上使用三轴 MM-RoPE。

每个轴使用 16 个频率。设第 j 个逆频率为 $\omega_j = \theta^{-j/16}$, 其中 $\theta = 10000$, $j = 0, \dots, 15$ 。时间、高度、宽度分别得到 $t\omega_j$ 、 $h\omega_j$ 、 $w\omega_j$ 三组角度, 一共 48 个角度。源码再把它们扩展为 96 维旋转参数, 所以 128 维 Attention Head 中前 96 维参与位置旋转, 剩余 32 维保持普通内容表示。

对任意二维子空间, RoPE 的旋转可以写成

$$R(\theta_p) \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} = \begin{bmatrix} \cos \theta_p & -\sin \theta_p \\ \sin \theta_p & \cos \theta_p \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}.$$

当两个位置 p 和 q 的旋转向量做内积时, 有 $R(\theta_p)^\top R(\theta_q) = R(\theta_q - \theta_p)$ 。因此 Attention 分数天然包含相对位置差。H3 把这个关系分别作用在时间、高度和宽度上, 所以模型既能区分视频中前后时刻, 也能区分上下和左右空间位置。

文本和音频也会被放入同一三维坐标体系。它们不一定拥有完整二维空间结构, 但仍然可以利用时间轴和预先分配的共享坐标与视频建立相对位置关系。

十一、时间嵌入：为什么生成模型必须知道当前噪声阶段

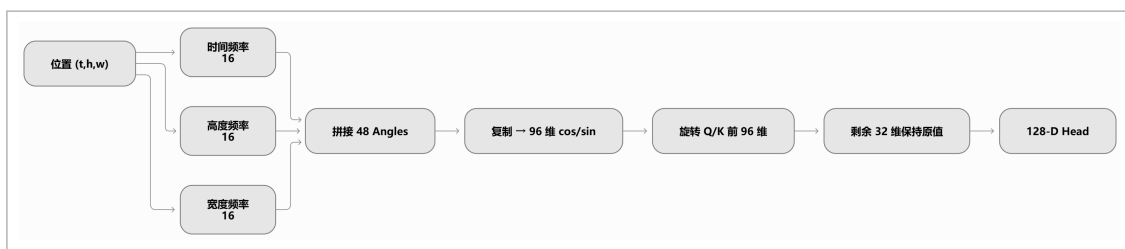


图 6 三轴 MM-RoPE 从 (t, h, w) 生成位置旋转。

Flow 模型在不同采样阶段面对的输入分布不同。早期 latent 接近随机噪声，后期 latent 已经包含较清晰的视频和声音结构。如果网络不知道当前时间 t ，同一个数值模式就无法判断应该做大幅修改还是小幅修正。

H3 先把 timestep 编码成 256 维频率向量，再经过 TimestepEmbedding 得到 2688 维时间表示 $e(t)$ 。一个请求中会同时出现多个 timestep，因为视频和音频使用两条独立 schedule，条件 Token 也可以固定在接近干净数据的位置。因此 Forward 会接收一组实际使用的 timestep，再通过每个 Token 的 timestep index 选择相应时间表示。

时间表示本身不会直接加到隐藏状态上。H3 把它送入每一层 AdaLN Projection，生成当前层需要的 shift、scale 和 gate。这样“当前处于什么去噪阶段”会影响每一层 Attention 和 FFN 的数值分布。

十二、AdaLN：为什么时间和模态要同时控制归一化

同一个 Transformer Block 同时处理文本、视频和音频，而且视频和音频还可能处于不同噪声时间。如果所有 Token 都使用完全相同的归一化参数，网络就要靠隐藏状态自己区分这些情况。H3 通过 AdaLN 直接把时间和模态信息加入每一层归一化过程。

对于某个 timestep，先得到 2688 维时间表示 $e(t)$ ，再计算

$$a(t) = W_{\text{ada}} \text{SiLU}(e(t)) + b_{\text{ada}}.$$

输出宽度是 96768，因为每层要为三种模态分别生成 Attention 与 FFN 的 shift、scale、gate，共 6 组 5376 维参数。

源码把这些参数按 timestep 和 modality 排列。对第 i 个 Token，若 timestep index 为 r_i ，modality tag 为 $m_i \in \{0, 1, 2\}$ ，则查表索引为 $k_i = 3r_i + m_i$ 。这样同一个视频 Token、文本 Token、音频 Token 即使共享同一层网络，也会取到不同的调制参数。

Attention 前的调制写成

$$\tilde{h}_i = \text{RMSNorm}(h_i) \odot (1 + s_{\text{msa},i}) + b_{\text{msa},i}.$$

Attention 输出写回残差时再乘 gate:

$$u_i = h_i + g_{\text{msa},i} \odot \text{Attn}(\tilde{H})_i.$$

FFN 部分同样使用自己的 $s_{\text{ffn}}, b_{\text{ffn}}, g_{\text{ffn}}$

scale 改变归一化后不同通道的幅值, shift 改变通道中心位置, gate 控制当前模块向 Residual Stream 写入多少更新。三者一起作用后, 同一套 Attention/FFN 可以在文本条件、视频去噪和音频去噪三种状态下使用不同的数值策略。

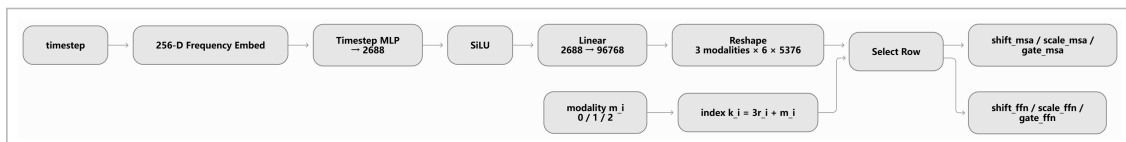


图 7 AdaLN 根据 timestep 与 modality 选择六组调制参数。

十三、为什么 AdaLN 会占约 13B 参数，又为什么推理时可以预计算

每层 AdaLN Projection 输入 2688 维, 输出 96768 维, 所以单层线性层就有约 260M 参数; 50 层合计约 13B。这个数量很大, 但它有一个特殊性质: 输出只依赖 timestep, 不依赖当前隐藏状态。

一次推理开始前, 视频 scheduler 和音频 scheduler 的全部 timestep 已经确定。假设整次采样一共只会出现 S 个不同时间点, 那么每一层所有需要的 $a(t)$ 都可以提前算好, 并缓存成一个调制表。主干运行时只需要按照 $k_i = 3r_i + m_i$ 查对应行。

因此 inference-only 部署可以先完成 AdaLN Projection 的预计算, 再只保留调制结果, 后续 50 层前向不再需要重复运行这些巨大线性层。Attention 与 FFN 仍然完整存在, 所以被移除的是“生成调制参数的网络”, 主干计算结构没有改变。

十四、Full Self-Attention: 文本、视频、音频如何真正发生交互

设第 ℓ 层 AdaLN 后的隐藏状态为 $\tilde{H} \in \mathbb{R}^{N \times 5376}$ 。Q、K、V 分别通过线性层映射到 7168 维, 然后 reshape 成 $N \times 56 \times 128$ 。每个 Head 都有独立的 Query、Key 和 Value。

Q 和 K 在计算点积以前先做 RMSNorm。对某个 128 维 Query q , 归一化为

$$\text{RMSNorm}(q) = \gamma \odot \frac{q}{\sqrt{\frac{1}{128} \sum_{j=1}^{128} q_j^2 + \epsilon}}.$$

这样做是为了控制 Query 和 Key 的整体幅值。Flow 去噪从纯噪声逐渐走向干净数据, 不同时间点的隐藏状态尺度会发生变化, QK Norm 可以降低这种尺度变化直接放大 Attention Logit 的影响。

随后对 Q、K 前 96 维加入 MM-RoPE，再计算

$$A_h = \text{softmax}\left(\frac{Q_h K_h^\top}{\sqrt{128}}\right), \quad O_h = A_h V_h.$$

分母 $\sqrt{128}$ 来自 Scaled Dot-Product Attention。若 Q、K 各维近似独立且方差相近，点积方差会随维度增加，因此除以 $\sqrt{d_h}$ 可以把 Logit 的数量级控制在更稳定范围。

当前开源实现使用非因果 Full Self-Attention。文本、视频和音频已经在同一个序列中，所以视频位置可以读取音频位置，音频位置也可以读取视频和文本位置。生成过程通过整段 latent 的 Flow 去噪完成，因此同一采样步中的目标位置可以双向交换信息。

56 个 Head 输出拼接后恢复为 7168 维，再通过输出投影回到 5376 维。最后由 AdaLN gate 决定这一层 Attention 更新向 Residual Stream 写入多少。

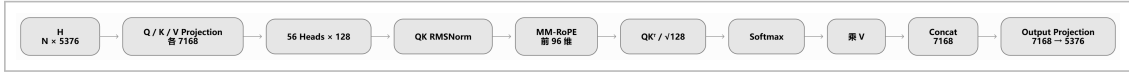


图 8 H3 全序列 Self-Attention 的完整计算。

十五、SwiGLU：为什么 Attention 后面还需要 FFN

Attention 主要完成不同 Token 之间的信息交换。一次 Attention 结束后，每个 Token 已经融合了其他位置的信息，但它仍然需要在自己的通道维度内部做更强的非线性变换，因此每层还要接 SwiGLU FFN。

对一个 5376 维 Token x ，先通过 Gate Projection 和 Up Projection 得到两个 14336 维向量 $g = xW_g$ 和 $u = xW_u$ 。Gate 分支经过 SiLU，再与 u 做逐元素乘法，最后通过 Down Projection 回到 5376 维：

$$\text{FFN}(x) = W_d [\text{SiLU}(W_g x) \odot (W_u x)].$$

这里的逐元素乘法使每一个中间通道都有一个由输入动态生成的门控值。SwiGLU 对三种模态共享同一套权重，模态差异由进入 FFN 前的 AdaLN 和输出 gate 控制。这样可以保持参数共享，同时让文本、视频和音频在同一层产生不同的激活模式。

十六、一个 Omni-Transformer Block 从头到尾怎样计算

现在把一层所有操作连起来。设输入为 $H^{(\ell)} \in \mathbb{R}^{N \times 5376}$ 。首先根据每个 Token 的 (r_i, m_i) 从 AdaLN 表中取出 Attention 的 scale、shift 和 gate，得到

$$\tilde{H}_a = \text{RMSNorm}(H^{(\ell)}) \odot (1 + S_a) + B_a.$$

随后执行统一 Self-Attention，并通过 gate 做第一次残差更新：

$$U = H^{(\ell)} + G_a \odot \text{Attn}(\tilde{H}_a).$$

接下来再根据同一组 timestep 和 modality 取 FFN 的 scale、shift 和 gate：

$$\tilde{U} = \text{RMSNorm}(U) \odot (1 + S_f) + B_f.$$

经过 SwiGLU 后得到第二次残差更新：

$$H^{(\ell+1)} = U + G_f \odot \text{FFN}(\tilde{U}).$$

这两个步骤不断重复 50 次。主隐藏形状始终保持 $N \times 5376$ ，文本、视频和音频始终处于同一条 Residual Stream 中。不同模态的差异主要体现在输入投影、模态标签、时间标签、位置坐标和最终输出头。



图 9 一个 H3-Omni-Transformer Block 的完整数据流。

十七、Rectified Flow：H3 训练和推理时到底预测什么

H3 的生成过程建立在 Rectified Flow 上。先设干净 latent 为 x_0 ，高斯噪声为 ϵ ，时间 $t \in [0, 1]$ ，其中 $t = 0$ 对应噪声端， $t = 1$ 对应干净数据端。最简单的直线路径写成

$$x_t = t x_0 + (1 - t)\epsilon.$$

对时间求导得到 $dx_t/dt = x_0 - \epsilon$ ，所以理想速度场是从噪声方向指向数据方向的 $x_0 - \epsilon$ 。

H3 scheduler 更常使用噪声量 $\sigma = 1 - t$ ，于是

$$x_t = (1 - \sigma)x_0 + \sigma\epsilon.$$

如果网络预测 data-ward velocity $v = x_0 - \epsilon$ ，将 $\epsilon = x_0 - v$ 代回去，可以得到 $x_t = x_0 - \sigma v$ ，于是干净 latent 的估计为

$$\hat{x}_0 = x_t + \sigma v_\theta(x_t, t, c).$$

这一步很重要，因为它直接说明网络输出的 velocity 怎样转成“当前对干净 latent 的估计”。H3 的公开 scheduler 使用这个方向定义，所以公式里是加号。

训练 Rectified Flow 时，网络需要让预测速度接近真实直线路径速度，可以写成

$$\mathcal{L}_{\text{RF}} = \mathbb{E}_{x_0, \epsilon, t} [\|v_\theta(x_t, t, c) - (x_0 - \epsilon)\|_2^2].$$

这个式子表达了 Rectified Flow 的基本训练原理。官方开源仓库主要公开推理权重和 scheduler，没有完整披露 H3 训练阶段所有损失项及其权重，所以这里不补充额外未公开项。

十八、Euler 更新：为什么一步可以写成 $x_{\text{next}} = x_t + \Delta\sigma v$

设当前噪声量为 σ ，下一步噪声量为 σ_{next} ，且 $\sigma_{\text{next}} < \sigma$ 。公开 scheduler 先定义 $r = \sigma_{\text{next}}/\sigma$ ，再计算

$$x_{\text{next}} = rx_t + (1 - r)\hat{x}_0.$$

将 $\hat{x}_0 = x_t + \sigma v$ 代入，可以化简为

$$x_{\text{next}} = x_t + (\sigma - \sigma_{\text{next}})v.$$

所以每一步更新只需要两个量：当前网络预测的方向 v ，以及当前 schedule 决定的步长 $\sigma - \sigma_{\text{next}}$ 。网络负责给方向，scheduler 负责决定这一小步走多远。

H3 的公开 scheduler 设置 $\eta = 0$ ，采样时不会额外加入随机噪声。给定初始 latent、条件和数值环境后，后续轨迹主要由固定时间表和 velocity 预测决定。

十九、为什么视频和音频要使用不同的 Flow Schedule

视频 latent 和音频 latent 的统计结构不同。视频在空间和时间上都有连续变化，音频是更高频的一维时间信号，因此两者适合的噪声时间分布不同。H3 为视频和音频分别使用一个 scheduler，公开视频 shift 为 12，音频 shift 为 3。

基础噪声量 σ 通过

$$\sigma' = \frac{s\sigma}{1 + (s - 1)\sigma}$$

重新映射。这个变换不会改变噪声量的顺序，但会重新分配不同区间中的采样密度。因为视频和音频使用不同的 s ，同一个采样步编号下两种模态可以对应不同的有效噪声量。

因此，H3 不能只用一个全局 timestep。每个视频 Token 需要知道视频 schedule 的当前位置，每个音频 Token 需要知道音频 schedule 的当前位置，条件 Token 还可能固定在接近干净数据的时间。timestep index 就是为了解决这个问题。

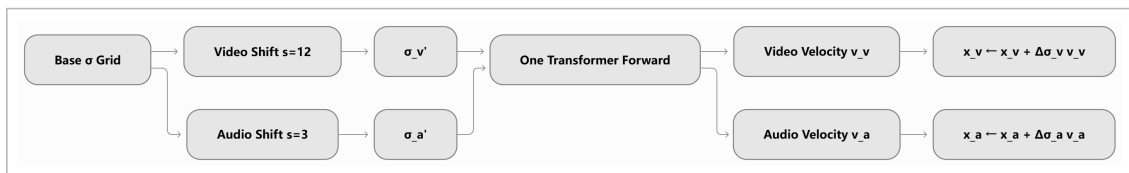


图 10 视频与音频的两条 Rectified-Flow 时间表。

二十、为什么视频和音频可以在同一次去噪里同步生成

在第 k 个采样步，视频当前 latent 记为 $X_v^{(k)}$ ，音频当前 latent 记为 $X_a^{(k)}$ 。两者通过输入投影写入同一条 Packed Sequence，再加入文本和参考条件。50 层 Self-Attention 会让视频位置读取音频位置，也让音频位置读取视频位置。

主干结束后，Video Head 得到视频 velocity $\hat{V}_v^{(k)}$ ，Audio Head 得到音频 velocity $\hat{V}_a^{(k)}$ 。两个 scheduler 分别更新

$$X_v^{(k+1)} = X_v^{(k)} + \Delta\sigma_v^{(k)} \hat{V}_v^{(k)},$$

$$X_a^{(k+1)} = X_a^{(k)} + \Delta\sigma_a^{(k)} \hat{V}_a^{(k)}.$$

两条更新的步长可以不同，但两个 velocity 都来自同一次跨模态 Transformer 前向。因此在每一个去噪阶段，画面变化都可以影响声音预测，声音条件也可以影响画面预测。同步关系在整个采样轨迹中反复更新，最后才分别交给两个 VAE Decoder。

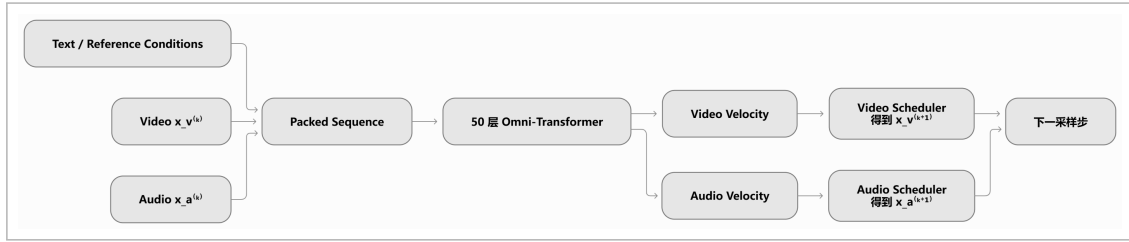


图 11 视频和音频在同一次 Transformer 前向中联合更新。

二十一、Final AdaLN 和两个输出头

50 层结束以后，整条序列还要经过 Final AdaLN。它继续根据 timestep 产生 scale 和 shift，但不再需要 residual gate。设最终隐藏状态为 $H^{(50)}$ ，则

$$\hat{H} = \text{RMSNorm}(H^{(50)}) \odot (1 + S_{\text{out}}) + B_{\text{out}}.$$

视频输出头把 5376 维映射到 96 维，因为每个视频 Token 对应一个 $24 \times 1 \times 2 \times 2$ Patch；音频输出头把 5376 维映射到 32 维。公开实现会先对整条序列做投影，再用 video indices 和 audio indices 取出目标行：

$$V_{\text{out}} = (\hat{H}W_v + b_v)[I_v], \quad A_{\text{out}} = (\hat{H}W_a + b_a)[I_a].$$

视频输出随后 Unpatchify 回 24 通道时空 latent，音频输出恢复成各自声道的 32 通道 latent，再交给对应 scheduler 更新。

二十二、首帧、尾帧和参考素材怎样成为固定条件

FL2VA 支持纯文本、首帧、尾帧和首尾帧约束。关键帧先通过 VisualVAE 编码成视觉 latent，再作为条件位置写入 Packed Sequence。公开 Pipeline 会把这些条件 latent 放在接近干净数据的时间位置，并在去噪循环中保持固定。

目标视频 latent 会在每一步被 scheduler 更新，条件关键帧不会随目标一起更新。因此关键帧可以在 50 层 Attention 中持续提供空间、人物、服装和场景信息，同时保持自己的视觉内容稳定。

Ref2VA 进一步加入参考视频和参考音频。参考视频同样通过 VisualVAE 编码，音频通过 AudioVAE 编码。它们都被分配到统一位置坐标中，后续每一层 Self-Attention 都能直接访问这些条件。

二十三、CFG Distillation：为什么推理时只需要一次主干前向

普通 Classifier-Free Guidance 会同时计算条件预测 v_c 和无条件预测 v_u ，再用 $v_{\text{cfg}} = v_u + w(v_c - v_u)$ 合成结果。这样每一个采样步骤通常需要两次大型 Transformer 前向。

H3 的开源 checkpoint 已经完成 Guidance Distillation。公开 Pipeline 没有 negative prompt 分支，也没有运行第二次无条件 Transformer。训练阶段把 guidance 后的行为蒸馏进单次条件模型，所以采样时直接使用 $v_\theta(x_t, t, c)$ 。

对 33B、50 层主干来说，这意味着每一步 Attention 和 FFN 的调用次数直接减少一半。视频和音频还共享同一次前向，所以联合生成没有再额外增加一套独立音频 Transformer。

二十四、Native Sparse Attention 为什么有必要

Packed Sequence 越长，Full Attention 的开销越大。设总 Token 数为 N ，每个 Head 都要计算 $N \times N$ 的 QK 相关性，主要复杂度与 N^2 成正比。长视频、更高分辨率和更多参考素材都会增加 N 。

如果稀疏 Attention 只允许每个 Token 连接 k 个有效位置，并且 $k \ll N$ ，相关性计算可以从近似 $O(N^2)$ 降到约 $O(Nk)$ 。H3 官方说明训练后期使用了 Native Sparse Attention，但首批开源推理代码仍然只给出 Full Attention 路径。

因此当前能够完整推导的是 Full Attention 的张量和复杂度。Sparse Attention 的精确连接规则、每个 Token 的邻接集合和实际稀疏比例还需要等待官方实现发布，讲义里不自行补充。

二十五、FL2VA 与 Ref2VA

H3-Base-FL2VA 面向纯文本、首帧、尾帧和首尾帧条件。不同任务共用相同的网络配置，差异主要体现在输入条件怎样编码、哪些视频位置固定、使用哪一组任务专用权重。

H3-Base-Ref2VA 面向更复杂的参考输入，可以同时接收多张图片、多段视频和参考音频。视觉参考一方面进入 Qwen3-VL 提供高层语义，一方面进入 VisualVAE 提供生成 latent；参考音频进入 AudioVAE。所有条件最终都进入同一条 Packed Sequence。

两组 checkpoint 都使用同样的 50 层、5376 hidden、56-head、14336-FFN Omni-Transformer 结构。它们的区别主要来自训练任务、条件组织方式和权重本身。

二十六、H3-Regenerate-2K 与当前开源边界

H3-Base 主要生成短边约 768 像素的结果。完整线上系统还包含 H3-Regenerate-2K，它会把基础结果与原始多模态上下文再次作为条件，继续生成 2K 视频。这样高分辨率阶段仍然可以读取原始文字、人物参考、视频参考和音频参考。

官方目前没有公开 H3-Regenerate-2K 的完整权重和内部网络，所以可以确认的是系统级流程：基础结果提供已有运动和画面结构，原始上下文继续提供语义与细节约束，最终重新生成高分辨率输出。内部层数、参数量和专用训练目标当前无法从公开代码确定。

H3-Context-IR 也属于完整系统的一部分。它负责解析复杂指令、跨模态关联和时间关系，但完整托管 workflow 没有随 H3-Base 一起开放。因此 H3-Base 可以进行源码级分析，Context-IR 和 Regenerate-2K 目前只能按照官方公开系统描述讲解。

二十七、完整过程：从输入条件到最终音视频

假设输入是一段文本、一张人物参考图和一段带声音的参考视频。文本和视觉参考先进入 Qwen3-VL-32B，H3 读取第 50 层 5120 维隐藏状态，投影到 5376 维并经过两层 Token Refiner。参考图和参考视频还会进入 VisualVAE，压缩成 24 通道时空 latent，再通过 $1 \times 2 \times 2$ Patchify 变成 96 维视频 Patch；参考音频通过 AudioVAE 从 32 kHz 波形变成 40 Hz、32 通道 latent。

目标视频和目标音频从随机噪声 latent 开始。Pipeline 把条件文本、参考视觉、参考音频、目标视频和目标音频写入同一条 Packed Sequence，同时为每个 Token 分配 modality tag、timestep index 和 (t, h, w) 坐标。MM-RoPE 根据位置坐标处理 Q/K，AdaLN 根据时间和模态选择当前层的 shift、scale 和 gate。

50 层主干反复执行 RMSNorm、AdaLN、Self-Attention、Residual、RMSNorm、

AdaLN、SwiGLU、Residual。因为三种模态始终处于同一条序列，所以每一层都能直接进行跨模态信息交换。

最后 Final AdaLN 处理整条序列，Video Head 输出 96 维视频 velocity，Audio Head 输出 32 维音频 velocity。视频 scheduler 和音频 scheduler 分别更新两种 latent，然后进入下一采样步。采样结束后，视频 latent 先 Unpatchify，再恢复通道 mean/std，通过 VisualVAE Decoder 还原成 24 FPS RGB 视频；音频 latent 恢复统计量后，通过 AudioVAE 解码左右声道，得到 32 kHz 立体声音频。

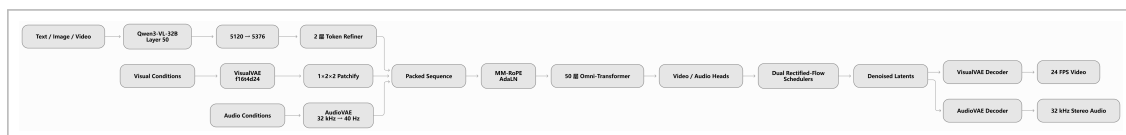


图 12 MiniMax H3-Base 从条件编码、联合去噪到音视频解码的完整数据流。

二十八、结构总结

MiniMax H3-Base 可以沿着四条关系理解。第一条是表示关系：文本从 5120 维投影到 5376 维，视频从 24 通道 latent 经过 Patchify 变成 96 维再投影到 5376，音频从 32 维投影到 5376，因此三种模态能够进入统一主干。

第二条是结构关系：每个 Token 同时带有模态标签、timestep index 和三维位置。MM-RoPE 处理位置，AdaLN 处理时间与模态，Self-Attention 处理 Token 之间的交互，SwiGLU 处理单个 Token 内部的通道变换。

第三条是主干关系：50 层 Transformer 始终保持 $N \times 5376$ 的隐藏形状。每层先做 AdaLN 条件化 Attention，再做 AdaLN 条件化 FFN。三种模态共享 Attention 和 FFN 权重，差异通过输入投影、位置、时间、模态调制和输出头保留下来。

第四条是生成关系：视频和音频使用两条 Rectified-Flow schedule，但每个采样步共享一次 Transformer 前向。网络分别预测视频和音频 velocity，scheduler 再按各自步长更新 latent。最终两个 VAE Decoder 分别恢复视频和双声道音频。

主要资料：

MiniMax H3 官方仓库：<https://github.com/MiniMax-AI/MiniMax-H3>

MiniMax H3 官方模型页：<https://huggingface.co/MiniMaxAI/MiniMax-H3>

MiniMax H3 官方介绍：<https://www.minimax.io/blog/minimax-h3>

Diffusers H3 Transformer：https://github.com/huggingface/diffusers/blob/main/src/diffusers/models/transformers/transformer_minimax_h3.py

Diffusers H3 Video VAE：https://github.com/huggingface/diffusers/blob/main/src/diffusers/models/autoencoders/autoencoder_kl_minimax_h3.py

Diffusers H3 Scheduler:https://github.com/huggingface/diffusers/blob/main/src/diffusers/schedulers/scheduling_minimax_h3.py